

Parameter name	Value	Parameter name	Value
Learning rate	0.001	Prior learning rate	0.01
Epochs	100	Amount beliefs prior	2500
Batch size	16	Prior initialization	0.1
# of samples	600	Prior iterations	50
Hidden layers	3	L2 on prior	900.000
Hidden width	800		

Table 4: Final hyperparameters for the multi-digit MNISTAdd task.

Parameter name	Value	Parameter name	Value
Perception Learning rate	0.00055	Prior learning rate	0.0029
Inference learning rate	0.003	Amount beliefs prior	2500
Batch size	20	Prior initialization	0.02
# of samples	500	Prior iterations	18
Hidden layers	2	L2 on prior	2.500.000
Hidden width	100		
Epochs	5000	Pretraining epochs	50

Table 5: Final hyperparameters for the visual Sudoku puzzle classification task.

We give the final hyperparameters in Table 4. We use this same set of hyperparameters for all N . # of samples refers to the number of samples we used to train the inference model in Algorithm 1. For simplicity, it is also the beam size for the beam search at test time. The hidden layers and width refer to MLP that computes each factor of the inference model. There is no parameter sharing. The perception model is fixed in this task to ensure performance gains are due to neurosymbolic reasoning (see [35]).

I.1.2 Other methods

We compare with multiple neurosymbolic frameworks that previously tackled the MNISTAdd task. Several of those are probabilistic neurosymbolic methods: DeepProbLog [35], DPLA* [37], NeurASP [58] and NeuPSL [44]. We also compare with the fuzzy logic-based method LTN [5] and with Embed2Sym [3] and DeepStochLog [56]. We take results from the corresponding papers, except for DeepProbLog and NeurASP, which are from [37], and LTN from [44]¹. We reran Embed2Sym, averaging over 10 runs since its paper did not report standard deviations. We do not compare DPLA* with pre-training because it tackles an easier problem where part of the digits is labeled.

Embed2Sym [3] uses three steps to solve Multi-digit MNISTAdd: First, it trains a neural network to embed each digit and to predict the sum from these embeddings. It then clusters the embeddings and uses symbolic reasoning to assign clusters to labels. A-NESI has a similar neural network architecture, but we train the prediction network on an objective that does not require data. Furthermore, we train A-NESI end-to-end, unlike Embed2Sym. For Embed2Sym, we use **symbolic prediction** to refer to Embed2Sym-NS, and **neural prediction** to refer to Embed2Sym-FN, which also uses a prediction network but is only trained on the training data given and does not use the prior to sample additional data

We believe the accuracy improvements compared to DeepProbLog to come from hyperparameter tuning and longer training times, as A-NESI approximates DeepProbLog’s semantics.

I.2 Visual Sudoku Puzzle Classification

I.2.1 A-NeSI definition

First, we will be more precise with the model we use. We see $\mathbf{x}$ as a $N \times N \times 784$ grid of MNIST images, and beliefs $\mathbf{P}$ as an $N \times N \times N$ grid of distributions over N options (for example, for a 4×4 puzzle, we have to fill in the digits $\{0, 1, 2, 3\}$). For each grid index i, j , the world variable $\mathbf{w}_{i,j}$ corresponds to the digit at location (i, j) . For correct puzzles, we know that the digits at location

¹We take the results of LTN from [44] because [5] averages over the 10 best outcomes of 15 runs and overestimates its average accuracy.

(i, j) and location (i', j') need to be different if $i = i', j = j'$ or if (i, j) is in the same block as (i', j') . For each pair of locations $(i, j), (i', j')$ for which this holds, we have a dimension in $\mathbf{y}$ that indicates if the digits at that grid location are indeed different. The symbolic function c considers each such pair and returns the corresponding $\mathbf{y}$.

For the prediction model, we use a *single* MLP f_θ . That is, for each pair that should be different, we compute $q_\phi(\mathbf{y}_k|\mathbf{P}) = f_\phi(\mathbf{P}_{i,j}, \mathbf{P}_{i',j'})$. This introduces the independence assumption that the digits at location (i, j) and location (i', j') being different does not depend on the digits at other locations. This is, clearly, wrong. However, we found it is sufficient to accurately train the perception model.

When training the prediction model, since we sample $\mathbf{P}$ from a Dirichlet prior that assumes the different dimensions of $\mathbf{w}$ are independent, the grid of digits $\mathbf{w}$ are highly unlikely to represent actual sudoku's: There are about 10^{21} Sudoku's and 9^8 possible grids (for 9×9 Sudoku's). However, it is quite likely to sample two digits that are different, and this is enough to train the prediction model.

I.2.2 Hyperparameters and other methods

We used the Visual Sudoku Puzzle Classification dataset from [4]. This dataset offers many options: We used the simple generator strategy with 200 training puzzles (100 correct, 100 incorrect). We took a corrupt chance of 0.50, and used the dataset with 0 overlap (this means each MNIST digit can only be used once in the 200 puzzles). There are 11 splits of this dataset, independently generated. We did hyperparameter tuning on the 11th split of the 9×9 dataset. We used the other 10 splits to evaluate the results, averaging results over runs of each of those.

The final hyperparameters are reported in Table 5. The 5000 epochs took on average 20 minutes for the 4×4 puzzles, and 38 minutes for the 9×9 puzzles on a machine with a single NVIDIA RTX A4000. The first 50 epochs we only trained the prediction model to ensure it provides reasonably accurate gradients.

While [4] used NeuPSL, we had to rerun it to get accuracy results and results on 9×9 grids.

We implemented the exact inference methods using what can best be described as Semantic Loss [57]. We encoded the rules of Sudoku described at the beginning of this section as a CNF, and used PySDD (<https://github.com/wannesm/PySDD>) to compile this to an SDD [28]. This was almost instant for the 4×4 CNF, but we were not able to compile the 9×9 CNF within 4 hours, hence why we report a timeout for exact inference. To implement Semantic Loss, we modified a PyTorch implementation available at <https://github.com/lucadiliello/semantic-loss-pytorch> to compute in log-space for numerically stable behavior. This modified version is included in our own repository. We ran this method for 300 epochs with a learning rate of 0.001. We ran this method for fewer epochs because it is much slower than A-NeSI even on 4×4 puzzles (1 hour and 16 minutes for those 300 epochs, so about 63 times as slow).

For both A-NeSI and exact inference, we train the perception model on *correct* puzzles by maximizing the probability that $p(y = 1|\mathbf{P})$. A-NeSI does this by maximizing $\log q_\phi(\mathbf{y} = \mathbf{1}|\mathbf{P})$, while Semantic Loss uses PSDDs to exactly compute $\log p(y = 1|\mathbf{P})$. For *incorrect* puzzles, there is not much to be gained since we cannot assume anything about $\mathbf{y}$. Still, for both methods we added the loss $-\log(1 - p(y = 1|\mathbf{P}))$ for the incorrect puzzles.

I.3 Warcraft Visual Path Planning

I.3.1 A-NeSI definition

We see $\mathbf{x}$ as a $N \times N \times 3 \times 8 \times 8$ real tensor: The first two dimensions indicate the different grid cells, the third dimension indicates the RGB color channels, and the last two dimensions indicate the pixels in each grid cell. The world $\mathbf{w}$ is an $N \times N$ grid of integers, where each integer indexes five different costs for traversing that cell. The five costs are $[0.8, 1.2, 5.3, 7.7, 9.2]$, and correspond to the five possible costs in the Warcraft game. The symbolic function c takes the grid of costs $\mathbf{w}$ and returns the shortest path from the top left corner $(1, 1)$ to the bottom right corner (N, N) using Dijkstra's algorithm. We encode the shortest path as a sequence of actions to take in the grid, where each action is one of the eight (inter-)cardinal directions (down, down-right, right, etc.). The sequence is padded with the do-not-move action to allow for batching.

For the perception model, we use a single small CNN f_θ for each of the $N \times N$ grid cells. That is, for each grid cell, we compute $\mathbf{P}_{i,j} = f_\theta(\mathbf{x}_{i,j})$. The CNN has a single convolutional layer with 6 output dimensions, a 2×2 maxpooling layer, a hidden layer of 24×84 and a softmax output layer of 24×5 , with ReLU activations.

The prediction model is a ResNet18 model [22], with an output layer of 8 options. It takes an image of size $6 \times N \times N$ as input. The first 5 channels are the probabilities $\mathbf{P}_{i,j}$, and the last channel indicates the current position in the grid. The 8 output actions correspond to the 8 (inter-)cardinal directions. We apply symbolic pruning (Section 3.3) to prevent actions that would lead outside the grid or return to a previously visited grid cell. We pretrained the prediction model by repeating Algorithm 1 on a fixed prior using 185,000 iterations (200 samples each) for 12×12 , and 370,000 iterations (20 samples) for 30×30 . We used fewer examples per iteration for the larger grid because Dijkstra’s algorithm became a computational bottleneck. This took 23 hours for 12×12 and 44 hours for 30×30 . Both used a learning rate of $2.5 \cdot 10^{-4}$ and an independent fixed Dirichlet prior with $\alpha = 0.005$. Standard deviations over 10 runs are reported over multiple perception model training runs on the same frozen pretrained prediction model. We trained the perception model for only 1 epoch using a learning rate of 0.0084 and a batch size of 70.

I.3.2 Other methods

We compare to SPL [1] and I-MLE [41]. SPL is also a probabilistic neurosymbolic method, and uses exact inference. Its setup is quite different from ours, however. Instead of using Dijkstra’s algorithm, it trains a ResNet18 to predict the shortest path end-to-end, and uses symbolic constraints to ensure the output of the ResNet18 is a valid path. Furthermore, it only considers the 4 cardinal directions instead of all 8 directions. SPL only reports a single training rule in their paper.

I-MLE is more similar to our setup and also uses Dijkstra’s algorithm. It uses the first five layers of a ResNet18 to predict the cell costs given the input image. One big difference to our setup is that I-MLE uses continuous costs instead of a choice out of five discrete costs. This may be easier to optimize, as it gives the model more freedom to move costs around. I-MLE is reported using the numbers from the paper, and averages over 5 runs.

To be able to compare to another scalable baseline with the same setup, we added REINFORCE using the leave-one-out baseline (RLOO, [29]), implemented using the PyTorch library Stochastic [53]. It uses the same small CNN to predict a distribution over discrete cell costs, then takes 10 samples, and feeds those through Dijkstra’s to get the shortest path. Here, we represent the shortest path as an $N \times N$ grid of zeros and ones. The reward function for RLOO is the Hamming loss between the predicted path and the ground truth path. We use a learning rate of $5 \cdot 10^{-4}$ and a batch size of 70. We train for 10 epoch and report the standard deviation over 10 runs. We note that RLOO gets quite expensive for 30×30 grids, as it needs 10 Dijkstra calls per training sample.

Finally, we experimented with running A-NEI and RLOO simultaneously. We ran this for 10 epochs with a learning rate of $5 \cdot 10^{-4}$ and a batch size of 70. We report the standard deviation over 10 runs.